

dwarven_engineering — System Design Spec

Dwarven Engineering — Overview

Problem statement

Dwarven Engineering is a 2.5D factory-builder colony sim with online co-op survival elements. Players join a persistent world hosted peer-to-peer by one of the players (with a lightweight cloud relay/rendezvous service for connection setup, not gameplay traffic) and build, craft, and survive together, with the world and colony state persisting across sessions on the host player's own machine. The differentiator is gameplay depth (colony sim complexity layered on survival/crafting/automated factory production), not infrastructure novelty — the P2P architecture exists to support that gameplay simply and cheaply, not to be the product's hook. Not a competitive game — no anti-cheat requirement (see 01-requirements.md).

Actors

- **Players** — join a shared persistent world in co-op groups, build/craft/manage a colony, survive environmental and creature threats. The only actor type in scope; no separate server-host/admin or live-ops actor role is modeled here (see non-goals and open questions).

Why now

The bet is on gameplay mechanics (colony sim depth combined with survival/crafting and persistence), not a novel infrastructure approach — competitors already do persistent worlds; this system needs to support that pattern well, not reinvent it.

Non-goals

- **No PvP/raiding** — purely cooperative; no player-vs-player combat or base destruction by other players.
- **No cross-platform play** — single platform at launch, no console/mobile cross-play infrastructure.
- **No user-generated mod marketplace** — mod support, if any, has no distribution/marketplace infrastructure in this design.

Dwarven Engineering — Requirements

Functional requirements

- **World join/persistence** — players join a persistent world; world state (terrain changes, structures, colony state) is saved and reloaded across sessions.
- **Building/crafting** — players place structures and craft items/tools from resources; colony infrastructure grows over time.
- **Colonist/NPC simulation** — AI colonists (distinct from players) have needs, jobs, and schedules — a colony-sim simulation layer beyond direct player actions.
- **Environmental/creature threats** — weather, hostile creatures, and resource scarcity create survival pressure players must respond to.

Read/write profile

Write-heavy, real-time — constant stream of position/state updates from all players and simulated entities (colonists, creatures), typical of a real-time multiplayer game server.

Non-functional requirements

- **Scale:** small co-op scale — 8-16 concurrent players per world session. Peer-to-peer means the company doesn't run or capacity-plan world instances at all; the scale question for company-owned infrastructure is concurrent Relay/Rendezvous connections and Auth throughput (total concurrent player population), not “how many worlds we host” — see 04-scaling.md.

- **Latency:** <50ms for gameplay-critical updates (movement, combat/interaction sync), over the direct peer-to-peer path between host and connected players once the Relay/Rendezvous service has brokered the connection. The company doesn't control this path's network quality (it's peer-to-peer, not routed through company datacenters) — the target shapes the *protocol* design (custom UDP, per 02-data-and-api.md), not a server placement decision.
- **Availability:** no company SLA on world-session availability — a world session is only as available as the host player's own machine and connection, which the company doesn't control or operate. Company-owned services (Auth, Relay/Rendezvous, Patch Delivery) do have their own availability targets: 99.9% (standard for lightweight, stateless cloud services), since these are conventional company-run infrastructure, unlike the P2P gameplay path.
- **Consistency:** strong consistency *within* a world session (the hosting player's client is the single authority — structures, inventory, colonist state all serialize through it); no cross-world consistency requirement since sessions are fully independent and ephemeral. Connected (non-host) players' client state may be momentarily stale/predicted and is reconciled against the host (standard client-prediction pattern for real-time action).
- **Security/compliance:** no compliance-driven constraints identified (no PvP, no cross-platform, no financial data in scope per 00-overview.md). **No anti-cheat requirement** — confirmed not needed, since this is a cooperative, non-competitive game where a cheating player only affects their own world session, not other players' outcomes.

Dwarven Engineering — Core Entities & API/Interface Design

Core entities

- **Player** — account/identity, persists across worlds and sessions.
- **PlayerCharacter** — in-world character state (position, inventory, stats) for a Player within a specific World.
- **Session** — an active peer-to-peer connection between a Player's client and a hosting Player's client, brokered by the Relay/Rendezvous service.
- **World** — a persistent world instance, owned entirely by the hosting player's client, which is the sole authority for that World's state and persists it to the host's local disk. This is the

consistency boundary (strong within a World session, no cross-World consistency needed); it is not company-hosted state.

- **Chunk** — a spatial subdivision of a World's terrain, used for streaming terrain to clients and scoping simulation/interest management.
- **Structure** — a placed building/construction within a World, owned by the hosting player's client.
- **Colonist** — an AI NPC entity with needs/jobs, simulated on the hosting player's client alongside the rest of the World (per the ownership decision below) — not a separate service.
- **Job** — a task assigned to a Colonist (or player-directed), driving Colonist behavior.
- **Need** — a Colonist's internal state (hunger, rest, etc.) that drives Job selection.
- **Item** — an item definition (type, stack size, etc.).
- **Inventory** — a container of Items, attached to a PlayerCharacter, Structure, or Colonist.
- **Recipe** — a crafting/production transformation from input Items to output Items.
- **Factory** (production building) + **TechNode** (tech tree) — automated production chains: Factories consume/produce Items via Recipes, connected by belt/pipe-style logistics that move Items between Factories automatically. TechNodes unlock new Factories/Recipes as the colony progresses. This automated item-flow simulation is a significant per-tick simulation load — flagged explicitly for 04-scaling.md, since it scales with colony complexity, not just player count.

Ownership

- **World/Chunk/Structure:** the hosting player's client owns all state in memory for its World, persisting periodically to local disk. There is no company-run server process and no shared DB — a DB round-trip per gameplay action would conflict with the <50ms latency target in 01-requirements.md regardless, but here it's moot: there's no company infrastructure in the gameplay data path at all once a session is connected.
- **Colonist simulation:** runs on the hosting player's client alongside the rest of the World simulation (not a separate service) — avoids a network sync boundary for a tightly-coupled simulation, at the cost of colonist AI competing for the same tick budget as everything else on that player's hardware (see 04-scaling.md for the implication of this alongside Factory/belt simulation — and note this is now the host's own hardware budget, not a company-scaled resource).

API / interface design

Real-time gameplay protocol: custom UDP protocol — resolved (previously WebSocket, see 07-risks-and-open-questions.md). Switched to actually guarantee the <50ms target in 01-requirements.md: TCP head-of-line blocking under packet loss would stall delivery of everything queued behind a single dropped packet, which a custom UDP protocol avoids by retransmitting/reordering only what's actually needed (e.g. reliable delivery for `player.action`, unreliable/latest-wins for `player.move` and `world.state_delta`). This is more implementation work than WebSocket but removes the latency tension entirely rather than accepting it as a launch risk.

Meta/non-realtime API: REST/JSON for account, auth, and player profile — separate from the UDP peer-to-peer gameplay connection used once a player joins a hosted world.

World discovery / connection setup: peer-to-peer via invite code, brokered by a lightweight Relay/Rendezvous service (NAT traversal + initial handshake) — not a full world directory service, since players connect to a specific host's session by invite code, not by browsing available worlds. Once the Relay/Rendezvous service has brokered the connection, gameplay traffic flows directly peer-to-peer (host ↔ each connected player), not through company infrastructure.

Auth: standard account auth (e.g. OAuth2/session token) via the REST API issues a token the client presents to the Relay/Rendezvous service and in the UDP handshake with the host, which validates it before admitting the player. Sessions are not re-validated after connect — see the accepted tradeoff in 06-fault-tolerance.md.

Versioning: UDP protocol version exchanged in the connection handshake; REST API versioned via URL path (`/v1/...`).

Key operations

Operation	Style	Payload (conceptual)	Notes
POST <code>/v1/auth/login</code>	sync REST	credentials → session token	Issued once, reused for Relay/host handshake
GET <code>/v1/players/{id}/profile</code>	sync REST	player id → profile, character list	Non-realtime
Relay broker request	sync REST/RPC	session token, invite code →	Relay/Rendezvous

Operation	Style	Payload (conceptual)	Notes
		host connection info	service resolves NAT traversal, hands the connecting client what it needs to reach the host directly
UDP connect + handshake (direct, peer-to-peer)	connection-oriented	session token, protocol version → accept/reject	Gate for joining the host's session once the Relay has brokered the connection; not re-checked after accept
player.move	unreliable UDP (peer-to-peer)	position/velocity delta	High-frequency, tick-rate bound; latest-wins, no retransmission needed
player.action	reliable UDP (peer-to-peer)	action type + target (build, craft, interact)	Host-authoritative — client predicts, host corrects; retransmitted until acked, since a dropped build/craft action can't just be superseded like a position update
world.state_delta	unreliable UDP (peer-to-peer, host→client)	changed entities since last tick	Sent per-tick to each connected player, scoped to their Chunk interest area; next tick's delta

Operation	Style	Payload (conceptual)	Notes
			supersedes a lost one

Idempotency is less relevant here than in a request/response API — the authoritative server resolves conflicting/duplicate client inputs each tick rather than relying on client-supplied idempotency keys. The reliable/unreliable split above is the UDP-specific equivalent: pick per-message-type delivery guarantees instead of get a single TCP-wide guarantee for everything.

Dwarven Engineering — Architecture

Legend:

[Name] generic service <[Name]> microservice / worker
 ((Name)) database ---> solid/pointed = sync call
 ..> dotted/pointed = async (event/queue) --- unpointed = peer/bidirectional

Component inventory

Generic services (company-run): Auth/Account Service, Relay/Rendezvous Service, Patch/ Content Delivery Service.

Microservices/workers (company-run): Analytics/Telemetry Worker — clients (host or connected players) push anonymized gameplay/crash telemetry directly to this worker; it no longer depends on a company-run World Server, since clients can emit events regardless of who’s hosting.

Peer-hosted (not company infrastructure): Host Client — the hosting player’s game client, which is the authoritative simulation for a World session (terrain, structures, colonists, factories/belts) and owns local persistence. Shown as a hexagon for diagramming consistency, but it runs on a player’s own machine, not company hardware — the company neither operates, provisions, nor scales it. This replaces the previous “World Server Fleet Manager + World Server process” design entirely (superseded, see 07-risks-and-open-questions.md).

Databases: Account/Profile DB (central, company-run), Analytics store (company-run). World save data lives locally on the Host Client's disk — not a company-run database (see Persistence below).

Split into two diagrams — account/connection setup, and per-session P2P runtime — rather than one dense graph.

Diagram 1 — Account & connection setup

Rendered from [03-architecture-diagram-1.json](#) via the skill's `render_diagram.py`; SVG version: [03-architecture-diagram-1.svg](#).

```
[ Client ]
  |-- login --> [ Auth/Account Service ]
  |-- patch/content check --> [ Patch/Content Delivery ]
  |.. anonymized telemetry ..> <[ Analytics/Telemetry Worker ]>
  |-- invite code + session token --> [ Relay/Rendezvous Service ]
  `-- direct P2P handshake (Relay steps out) --> <[ Host Client
(peer) ]>

[ Auth/Account Service ] -- reads/writes --> (( Account/Profile
DB ))
<[ Analytics/Telemetry Worker ]> -- writes --> (( Analytics
store ))
[ Relay/Rendezvous Service ] -- brokers NAT traversal --> <[ Host
Client (peer) ]>
```

The Client fans out to every company-run entry point: telemetry is dotted (async, never gameplay-blocking); the direct P2P handshake branch is the moment the Relay steps out of the path and the Client talks straight to the Host peer.

Diagram 2 — Per-session P2P runtime

Rendered from [03-architecture-diagram-2.json](#) via the skill's `render_diagram.py`; SVG version: [03-architecture-diagram-2.svg](#). The Host Client (terrain, structures, colonists, factories/belts) ticks on the hosting player's own machine and fans out state deltas over custom UDP to every connected player, plus periodic local snapshots.

```
<[ Host Client (peer) ]>
  |-- custom UDP, direct P2P (state delta) --> [ Connected Player
1 ]
  |-- custom UDP, direct P2P (state delta) --> [ Connected Player
2 ]
  |-- custom UDP, direct P2P (state delta) --> [ Connected Player
```

N (up to 8-16)]

`-- periodic snapshot --> ((Local save file (host disk)))

Notes: - No company infrastructure is in the real-time gameplay path — Relay/Rendezvous only brokers the initial connection, then steps out; the Auth/Account Service and Patch Delivery Service are entirely off the gameplay path. This is a stronger version of the same principle from the previous design (nothing latency-critical touches company infra), just with the authoritative component itself now also off company infra. - The Host Client is a single point of failure by construction — if the host disconnects, the session ends for everyone (confirmed, not host-migrated); see 06-fault-tolerance.md. - Local save file is not a company-run database — it's shown as a circle for diagram consistency (it's still "a database" conceptually — persistent structured state) but it lives entirely on the host player's own disk, with no company backup or redundancy by default.

CAP theorem tradeoffs

Store	CP or AP	Why
Account/Profile DB	CP	Login/profile data must be consistent — no conflicting account state across replicas. Low write volume, so CP is cheap here. Only one writer (the Host Client) ever touches it — there's no distributed consistency question at all, since there are no replicas or concurrent writers to reconcile.
Local save file (Host Client)	CP , single-writer	Not gameplay-critical — losing or delaying a few telemetry events during a partition is an acceptable tradeoff for availability; must never be gameplay-blocking, especially now that it's the
Analytics store	AP	

Store	CP or AP	Why
		only remaining company-run write path touched during a session.

Note: with company-run World Server processes gone, most of the previous CAP discussion (World Save store CP tradeoff, Analytics store AP tradeoff) no longer applies — the company owns far less stateful infrastructure in this design. The interesting availability question is now entirely about the Host Client (a single peer’s machine), which is a fault-tolerance concern (see 06-fault-tolerance.md), not a CAP tradeoff the company can engineer around.

Dwarven Engineering — Scaling Strategy

Per-session simulation scaling (Host Client)

Colonists, factories/belt-based item-flow simulation, and structures all tick on the hosting player’s own client — this is no longer a company scaling problem, since the company doesn’t operate or provision this compute at all (superseded from the earlier company-hosted design; see 07-risks-and-open-questions.md). The factory/belt simulation load flagged in 02-data-and-api.md is now entirely the host’s own hardware budget.

What the company *can* still do: ship the client with the same tick-budget-management approach previously designed for company-run servers (e.g. degrade non-critical simulation detail — far-from-player factory tick frequency — before letting tick rate slip), just running on the host’s machine instead of company infrastructure. This is a client engineering concern, not a capacity planning one.

Company infrastructure scaling (Auth, Relay/Rendezvous, Patch Delivery, Analytics)

These are the only components the company operates and needs to capacity-plan:

- **Auth/Account Service:** scales with total concurrent player population (login/profile requests), not with number of active world sessions — standard stateless service, horizontally scalable, backed by the Account/Profile DB.
- **Relay/Rendezvous Service:** scales with *connection setup* rate (new sessions being joined), not with ongoing gameplay traffic — it steps out of the path once a P2P connection is established (per 03-architecture.md), so its load profile is much lighter than a service that stayed in the gameplay path.
- **Patch/Content Delivery Service:** standard CDN-style scaling, independent of the other services.
- **Analytics/Telemetry Worker:** scales with total client population's event rate; async, never gameplay-blocking (per 03-architecture.md).

None of these need the bin-packing/fleet-management scaling strategy from the earlier company-hosted design — that entire concern (World Server Fleet Manager, host bin-packing, per-world resource rebalancing) is superseded, not merely simplified.

Regional placement

Relay/Rendezvous Service can run in one region at launch (or a small number of regions) since its job is brief connection brokering, not sustained gameplay traffic — its own latency sensitivity is much lower than the peer-to-peer gameplay path it sets up. Multi-region here is a cost/reach tradeoff for the company's own infrastructure, not a factor in gameplay latency (which is governed by the direct peer-to-peer path's own network route, outside company control).

Bottleneck watchlist

- **Host Client factory/belt simulation growth:** still the single biggest risk to a session's playability as a colony matures — no longer a company-side bottleneck to watch via internal dashboards, but a client-side performance ceiling worth surfacing to players (see 05-observability.md for how visibility into this now works without company-run per-session telemetry).

- **Relay/Rendezvous connection-setup rate:** the one company-run component whose load actually scales with total active sessions (each new join is a brokering event) — worth watching as player population grows, even though it's a much lighter load profile than hosting gameplay itself would be.

Dwarven Engineering — Observability & Telemetry

What changed from the company-hosted design

The previous design's highest-priority signal — per-world tick time on a company-run World Server, alerted and paged on — no longer applies. The company doesn't operate the Host Client, so there's nothing to page an on-call engineer about when a specific session's simulation slows down; that's the host player's own hardware, not company infrastructure. Observability now splits cleanly into two categories: RED/USE for the company's own services (still applies, lower stakes than before), and client-reported telemetry (informational, not actionable via paging).

RED metrics (Rate, Errors, Duration) — per company-run service

- **Auth/Account Service:** standard RED tracking (request rate, error rate, response duration).
- **Relay/Rendezvous Service:** rate = connection-brokering attempts/sec, errors = failed handshakes/NAT-traversal failures, duration = time to broker a connection. This is now the closest thing to a “gameplay-adjacent” SLO the company owns, since a slow/broken Relay directly delays players joining a session even though it's not in the ongoing gameplay path.
- **Patch/Content Delivery Service:** standard CDN RED metrics.
- **Analytics/Telemetry Worker:** RED tracked but non-critical — never allowed to apply backpressure to any client (per 03-architecture.md).

USE metrics (Utilization, Saturation, Errors) — per resource

- **Account/Profile DB, Analytics store:** standard connection pool/CPU/disk/replication-lag USE metrics — same as any conventional cloud service.
- **Relay/Rendezvous Service:** connection-handling capacity (concurrent brokering sessions), since a spike in players joining at once (e.g. after a content update) is its main saturation risk.
- **Host Client simulation performance:** no longer a company-operated USE metric — see client telemetry below for how the company gets *any* visibility into this without owning the infrastructure.

Client-reported telemetry (replaces per-world dashboards)

Per-world dashboards from the earlier design don't make sense here — there's no company-run per-session infrastructure to dashboard. Instead: Host Clients can opt in to reporting anonymized performance telemetry (tick time, factory/colonist counts, frame time) to the Analytics/Telemetry Worker, giving the company aggregate visibility into “how often do colonies of size X start struggling” without operating or monitoring any individual session live. This is informational for game-balance/performance-tuning decisions, not an operational SLO — nothing here gets paged, since there's no company action to take on an individual struggling session.

Logging and tracing

Client-side structured logs (tagged with an anonymized session id) for crash reports and correlation of a specific player action to a reported bug — sent to the Analytics/Telemetry Worker, not a company-run distributed trace across services (there's no multi-service gameplay path left to trace; the Host Client is a single process).

SLOs and alerting

- **Relay/Rendezvous Service:** p95 connection-brokering time under a target (e.g. a few seconds), error rate < 1% on handshake attempts — paging on sustained breach, since this is the one company-run component that directly affects a player's ability to join a session.

- **Auth/Account Service:** standard p95 latency / error-rate SLOs, paging on sustained breach.
- **Company service availability:** 99.9% for Auth, Relay/Rendezvous, and Patch Delivery (matches 01-requirements.md) — there is no equivalent SLO for world-session availability, since that's the host player's own machine, not company infrastructure.

Dwarven Engineering — Fault Tolerance & Resilience

What changed from the company-hosted design

The previous design's central concern — restarting a crashed company-run World Server from a snapshot — no longer applies, since there's no company-run World Server. The Host Client is the host player's own machine; the company can't detect its crash, restart it, or restore its state. Fault tolerance now splits into: (a) what happens to a session when the Host Client fails, which is a product/UX design question, not an infra one, and (b) fault tolerance for the company's own services, which is conventional and lower-stakes than before.

Per-component failure modes

- **Host Client (single authority per World session):** a single point of failure by design — not mitigated, **accepted**. If the hosting player's client crashes, disconnects, or closes the game, **the session ends for everyone** (confirmed — no host migration). Connected players are dropped and must wait for the host to relaunch and resume from their local save. This is a deliberate simplicity choice: host migration (transferring authority to another peer mid-session) would require real-time state handoff and is significant added complexity for a cooperative, non-competitive game where "the host left, session's over for now" is an acceptable and easily-understood outcome to players.
- **Local save file (on Host Client's disk):** persisted periodically by the Host Client itself, with **no company backup** — if the host's disk fails or the save is corrupted, that world's progress is lost unless the player has their own backup. This is a real player-facing risk the company doesn't mitigate by default; flagged in 07-risks-and-open-questions.md

as worth a product decision (e.g. optional cloud save sync) even though it's out of scope for now.

- **Auth/Account Service / Account/Profile DB:** unchanged in spirit from the earlier design — auth is checked at Relay/Rendezvous and host-handshake time, not re-validated mid-session, so an Auth outage doesn't disconnect already-playing players, only blocks new logins/joins.
- **Relay/Rendezvous Service:** if it's down, players can't set up *new* P2P connections (can't join a session or start one), but **already-connected sessions are entirely unaffected** — it's out of the gameplay path once brokering is done (per 03-architecture.md), which is an even cleaner story than the earlier Fleet Manager's "not in the gameplay path" property, since there's no restart-detection role for it to lose either.
- **Analytics/Telemetry Worker:** can fail or fall behind with zero impact on gameplay, by design (async, non-blocking, per 03-architecture.md).

Retries, timeouts, backoff

- **Connecting Client → Relay/Rendezvous:** retried with backoff on a failed brokering attempt (e.g. transient NAT-traversal failure) — standard client-side retry, no company-side escalation needed since this doesn't affect anyone already in a session.
- **Connected Player → Host Client (P2P):** client-side reconnect with backoff on disconnect — but note a reconnect only succeeds if the Host Client is still up; if the host itself is gone, reconnect attempts fail until the host relaunches (per the accepted "session ends" behavior above).

Redundancy / bulkheads

- Each World session is its own bulkhead by construction — one host's crash or overloaded simulation cannot affect any other session, since there's no shared process or shared infrastructure between sessions at all (stronger isolation than the earlier bin-packed design, since sessions don't even share a host machine).
- Company-run services (Auth, Relay/Rendezvous, Patch Delivery, Analytics) are independent of each other and of any individual session — standard multi-instance redundant deployment, nothing session-specific to bulkhead.

Degradation paths

- Host Client crashes/disconnects → session ends for everyone; no effect on any other session.
- Auth/Account Service down → already-connected sessions keep playing; new logins/joins blocked until it recovers.
- Relay/Rendezvous Service down → already-connected sessions keep playing; new sessions/joins blocked until it recovers.
- Analytics Worker down → zero player-facing impact; only telemetry/analytics data is delayed or lost.

Single points of failure

- **Host Client** is an accepted SPOF per session, by design, with no mitigation beyond “the session ends and the host can relaunch” — replicating or migrating a peer-hosted authoritative simulation is out of scope given the genre (cooperative, non-competitive, small groups who understand “the host left”).
- **Relay/Rendezvous Service** is a SPOF for new connection setup only, not for already-running sessions — standard redundant deployment (multi-instance) is sufficient.

Disaster recovery (RTO/RPO)

- **RPO/RTO for a World session:** not applicable in the traditional sense — there is no company- operated recovery process. The host’s own local save frequency determines their personal data- loss window on a crash, and “recovery” is simply relaunching the game — this is a client UX concern (e.g. autosave frequency, save-file rotation to guard against corruption), not a company infrastructure DR plan.
- **RTO/RPO for company-run services** (Auth, Relay/Rendezvous, Patch Delivery, Analytics): conventional cloud-service DR — standard managed redundancy is sufficient given these are stateless or low-write-volume services; no aggressive target needed since none of them hold session-critical state.

Dwarven Engineering — Risks, Assumptions & Open Questions

Collected from every section of the interview, including a major architecture pivot partway through (company-hosted World Servers → peer-to-peer with a lightweight Relay/Rendezvous service). None of these were silently resolved — each either has an explicit decision recorded, is superseded by the pivot, or is still open and flagged as such.

Superseded by the peer-to-peer pivot (no longer applicable)

These were resolved decisions in the earlier company-hosted design; the architecture change made them moot rather than requiring a new resolution:

- **World Server Fleet Manager, bin-packing, host rebalancing policy** — there is no company-run World Server to provision, bin-pack, or rebalance. Superseded.
- **Fleet Manager restart RTO (under 2 minutes)** — there is no company-run process to restart. The equivalent question now is client-side autosave/reload UX, not an infra RTO — see the new open item below.
- **Per-world 99.5% availability SLA** — replaced by: no company SLA on session availability at all (it's the host player's own machine); company-run services now target 99.9% instead. See 01-requirements.md.

Resolved (from the original four open questions, still applicable post-pivot)

1. **WebSocket vs the <50ms latency target — RESOLVED:** custom UDP protocol, now used for the direct peer-to-peer path (host ↔ connected players) rather than client↔company-server. See 02-data-and-api.md, 03-architecture.md.
2. **Single-region vs the <50ms target — RESOLVED, and simplified by the pivot:** moot for gameplay traffic, since that's now a direct peer-to-peer path outside company control; the Relay/Rendezvous service (the one latency-sensitive company component left) can run single-region at launch since its role is brief connection brokering. See 04-scaling.md.

3. **Session revalidation gap — RESOLVED/ACCEPTED AS-IS:** unchanged by the pivot — sessions are validated at connect time only. See 06-fault-tolerance.md.
4. **Anti-cheat requirement — RESOLVED: not needed.** Confirmed non-competitive, cooperative game — a cheating player only affects their own world session. See 01-requirements.md.

New open questions from the peer-to-peer pivot

1. **No company backup for local save files** (06-fault-tolerance.md). If the host's disk fails or a save corrupts, that world's progress is lost with no company-side recovery path. **Needs a product decision:** accept this as a player-owned-risk model, or add optional cloud save backup/sync (which would reintroduce some company-run persistence infrastructure).
2. **Relay fallback for restrictive NAT/firewalls** (02-data-and-api.md, 03-architecture.md). The design assumes the Relay/Rendezvous service can broker a direct P2P connection via NAT traversal, but some networks (symmetric NAT, restrictive firewalls) can't establish direct P2P at all and need a full traffic-relaying fallback (TURN-style), not just a brokering handshake. **Needs a decision:** is a fallback relay path in scope, or is "some players can't connect to some hosts" an accepted limitation?
3. **Host Client autosave/reload UX** (06-fault-tolerance.md). Replaces the old Fleet-Manager- restart-RTO question — now a client design question: how often does the Host Client autosave locally, and what's the reload flow after a crash? Not specified in this interview.

Still open — assumptions carried over from before the pivot

1. **Bin-packing rebalancing policy** — superseded, see above (no longer applicable).
2. **Host-level blast radius from bin-packing** — superseded, see above (no longer applicable; each session is now isolated to its own host player's machine by construction).

Non-goals confirmed (not risks, just recorded for traceability)

- No PvP/raiding between colonies (00-overview.md).
- No cross-platform play (00-overview.md).
- No user-generated mod marketplace (00-overview.md).
- No anti-cheat (01-requirements.md, confirmed this session).